

Agentic Systems (Multi -Agents)

An LLM can only talk. An agent can do.

An agent is an LLM that's been given hands — it doesn't just tell you how to do something, it actually goes and does it for you.

LLM = answers your question.

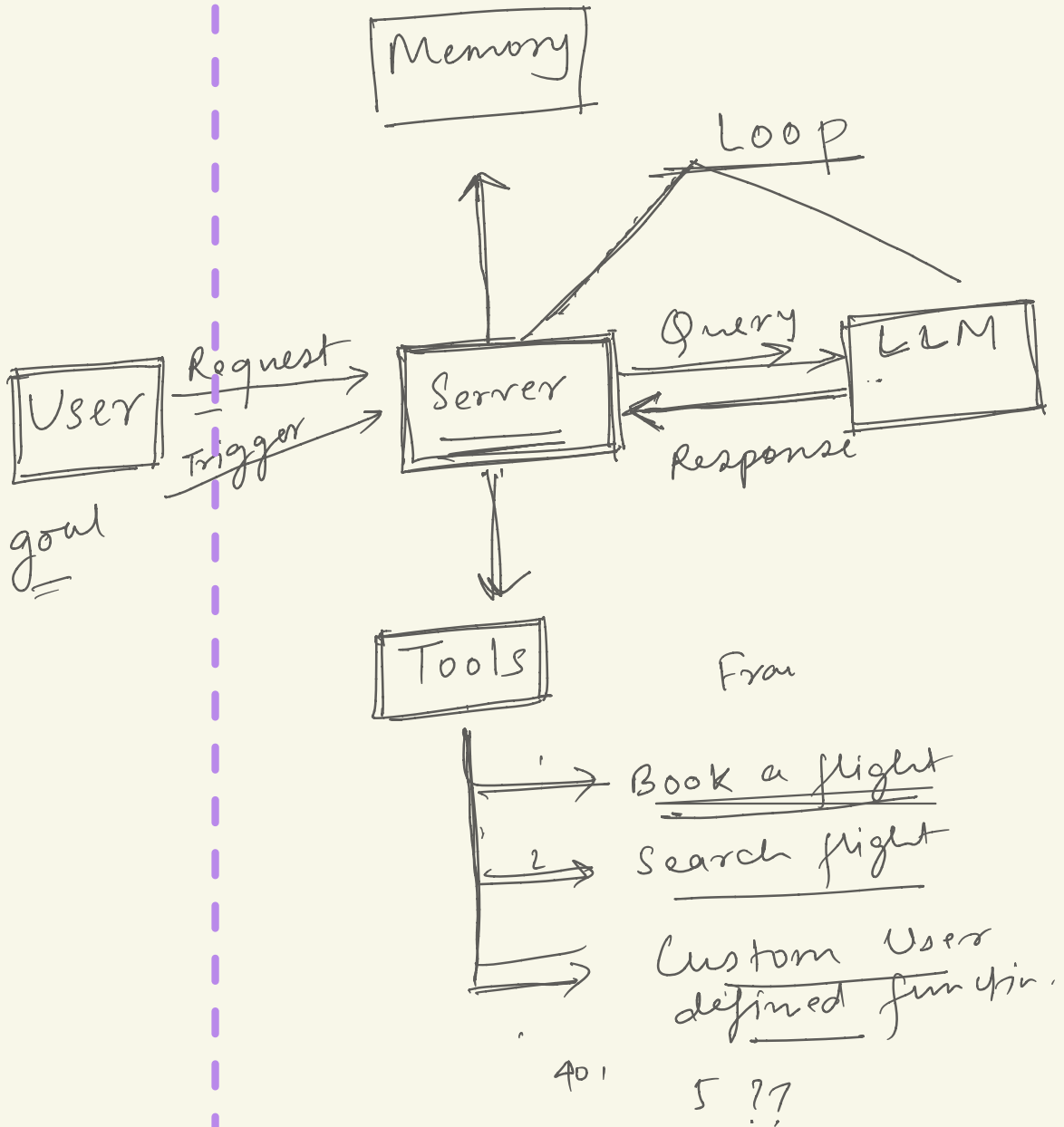
Agent = completes your task.

User prompt	Plain LLM response (just talks)	Agent response (takes action)
"Book me a table at 8 PM."	"Sure! You can call the restaurant or use OpenTable to book."	<i>Actually books the table and replies "Done — table for 8 PM, confirmed."</i>
"What's the weather tomorrow?"	"I can't check live weather, but you can look on http://weather.com ."	<i>Calls a weather API and says "It'll be 12°C and rainy."</i>
"Email colleague the report."	"Here's a draft email you can copy and send to John."	<i>Sends the email to somebody and replies "Sent."</i>
"Reply to my unread messages."	"Here's what you could say to each person."	<i>Opens your inbox, reads them, and sends the replies.</i>

LLM → "Here's HOW you book a flight." (read / fetch / explain)

Agent → "I booked your flight. Confirmation: ABC123." (do it for you)

Agent



def. json schema → descriptio

That agent works – but it has three real-world weaknesses:

1. **Adding tools is painful.** Every new capability means hand-writing a tool definition.
2. **One brain doesn't scale across domains.** A single agent juggling fraud detection **and** customer comms **and** refunds gets confused.

→ A bigger prompt with more unrelated tools = more chances to pick the wrong one and a higher cost.

3. **It forgets** Close the chat and your agent has amnesia.

The agenda

1. **Model Context Protocol (MCP)** → how agents discover & use tools at scale
2. **Multi-Agents** → many specialised agents working together
3. **Memory** → remembering within and across conversations

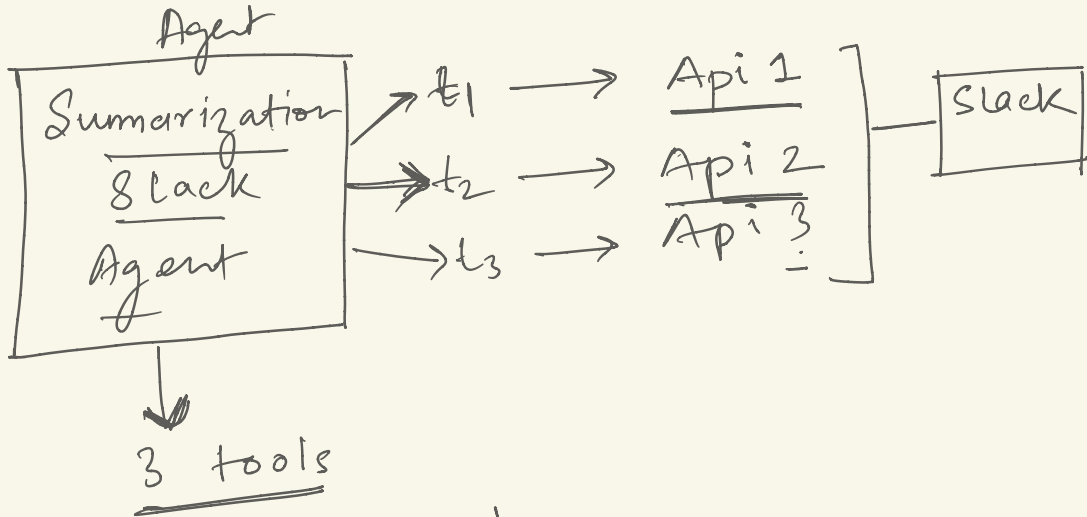
Model Context Protocol – MCP

Tools don't scale

Say we want our agent to use **Slack** (send a message, read a channel, search past messages).

Slack exposes its functionality as a REST API.

For **every Slack API** our use case needs, we hand-write a tool definition and the code to call it.



- ① Read a slack channel
- ② Search messages
- ③ Send messages.

Step 1 – write the real function (lives on the server):

```
def send_slack_message(channel: str, text: str):  
    # the actual work: hit Slack's API and post the message  
    return {"status": "sent", "channel": channel}
```

Step 2 – describe it as a tool (the schema the LLM reads):**

```
tool = {  
    "name": "send_slack_message",  
    "description": "Post a message to a Slack channel. Use when  
the user "                "asks to notify or message someone on  
Slack.",  
    "parameters": {  
        "channel": "string – e.g. '#general'",  
        "text":    "string – the message to post",  
    },  
}
```

Step 3 – pass the tool in when you call the LLM:**

```
response = llm.chat(  
    messages=[{"role": "user", "content": "Tell #general we  
shipped!"}],  
    tools=[tool],          # ← the schema rides along in the  
    request  
)
```

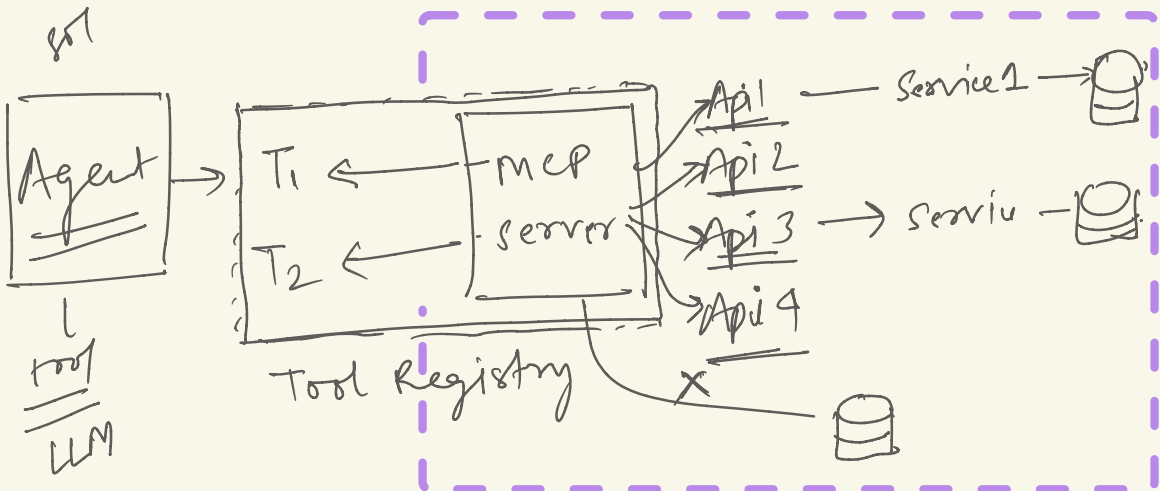
The drawbacks:

- (i) Tedious to code → one tool wrapper per API endpoint
- (ii) Brittle → Slack ships a new/changed API → our code breaks
- (iii) Developer burden → WE define every tool, not the people who own the API

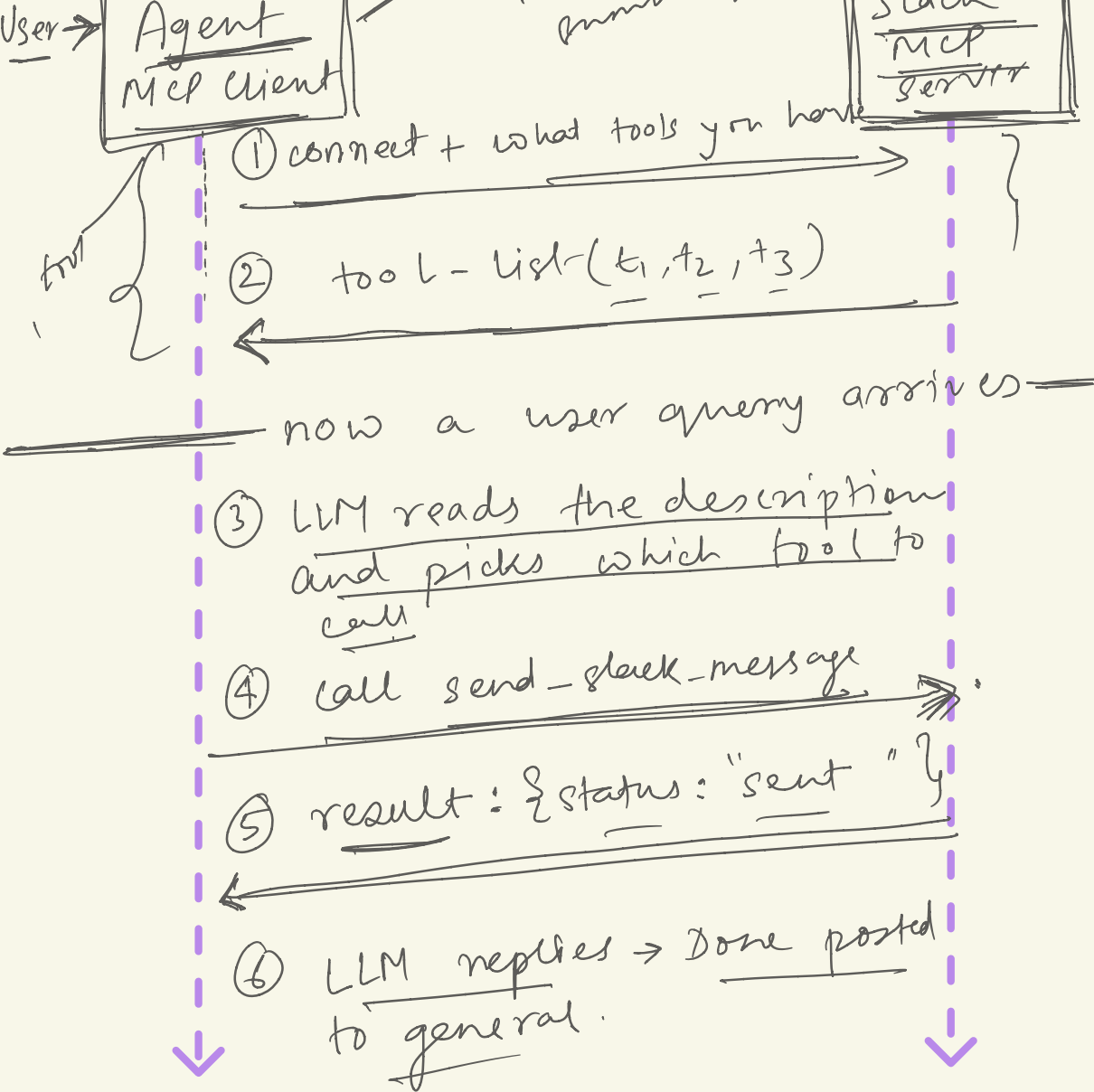
- > 1. **What** does this tool do?
- > 2. **When** is it used? (triggers, conditions)
- > 3. **What** each parameter means.
- > 4. **What** the tool **returns**.
- > 5. **Examples** of input parameters.

Instead of *us* wrapping Slack's APIs,

Slack stands up an **MCP server** that exposes its own capabilities as ready-made tools.



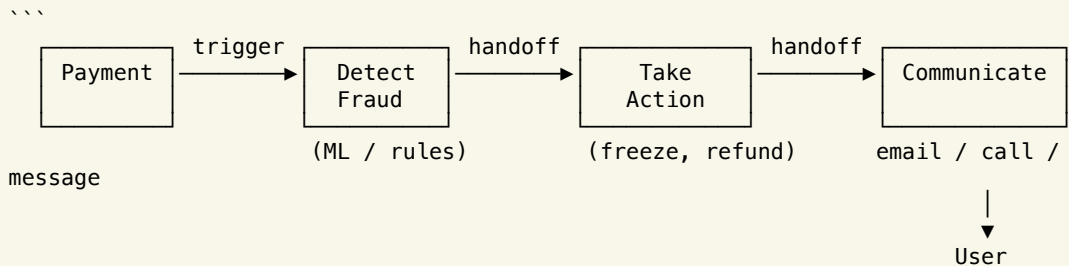
The workflow



Multi-Agents

A single agent is perfect for a single-domain task.

Now consider the full lifecycle of a flagged payment. It crosses *several* domains



Detecting fraud, deciding the action, and *communicating with the customer* are genuinely different skills with different tools.

Putting all of it into one agent gives you a confused generalist with 40 tools and a bloated prompt.

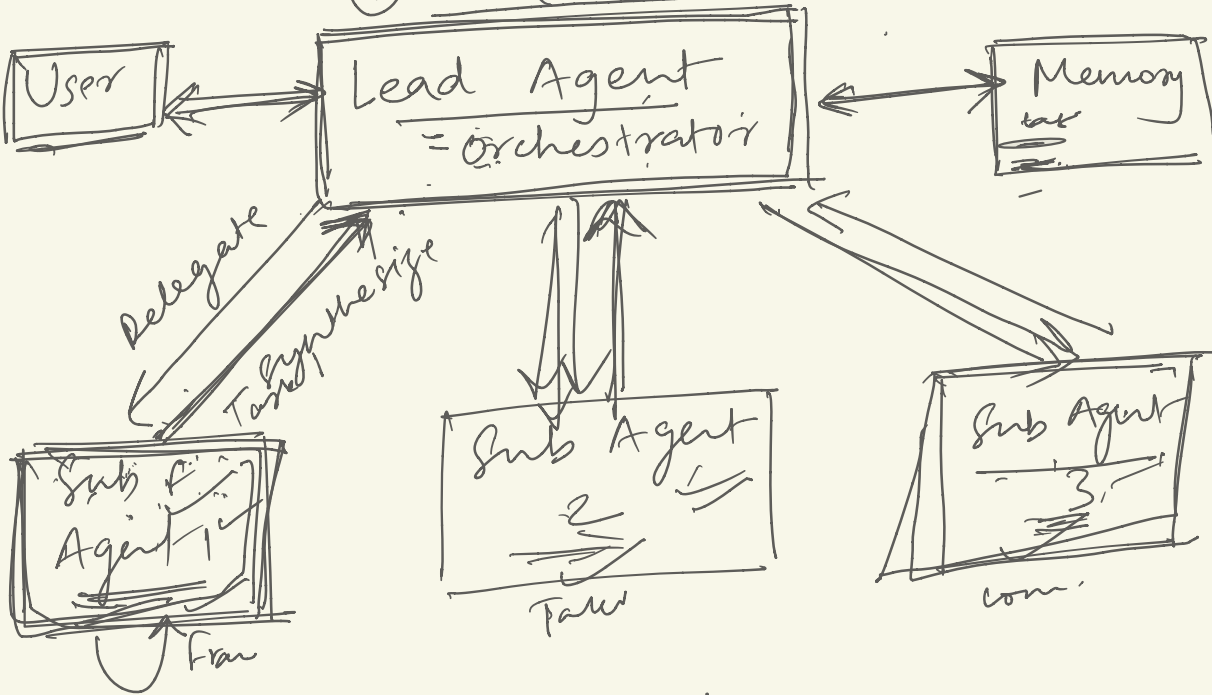
For these kind of use cases we require agents with specific domain expertise . The agents are independent with separate context management and specialised tools for their task

Anthropic reported that a system of *Opus 4* (leader) + *Sonnet 4* (sub-agents)

outperformed a single *Opus 4* agent on deep-research tasks.

Orchestrator Design Pattern

① Plan (Written to Memory)



The flow:

1. **Plan** the lead agent breaks the goal into sub-tasks and writes the plan to memory.
2. **Delegate** – it hands each sub-task to a specialized sub-agent.
3. **Sub-agents** run their own loops – each is a full agent (its own Think → Act → Observe) with its own tools.
4. **Synthesize / aggregate** – sub-agents report results back; the lead combines them and returns the final answer to the user.

USER: "There's a \$2,000 charge I didn't make!"

① LEAD AGENT plans:

"I need to (a) check if it's fraud, (b) act on it, (c) tell the customer."

② Delegates to FRAUD AGENT:

→ checks transaction history + risk score

→ reports back: "High risk. Charge is in another country. Likely fraud."

③ Delegates to ACTION AGENT:

→ freezes the card + starts a refund

→ reports back: "Card frozen. Refund of \$2,000 initiated, ETA 3 days."

④ Delegates to COMMS AGENT:

→ drafts a clear, friendly message

→ reports back: "Customer notified via email + SMS."

⑤ LEAD AGENT synthesizes everything and replies to the user:

"We confirmed the charge was fraudulent, froze your card, and started a \$2,000 refund (arriving in ~3 days). A confirmation is in your email."

Multi-agent systems are hard (Disadvantages)

...

1. Task delegation → Is the lead agent actually good at splitting work well?

2. Isolation vs sharing → Sub-agents need their own context, BUT sometimes need to share findings. Getting this balance right is hard.

3. Lead assessment → Can the lead correctly judge if a sub-agent's result is good enough, or does it accept garbage?

4. Cost → Anthropic reported ~15× token usage vs a single agent.

5. Debugging → Non-deterministic: same prompt → different paths each run. Reproducing a failure is genuinely hard.

...

—

Memory

What memory actually is

Memory is the history of an agent's decisions, tool calls, results, and feedback.

The trade-off is simple and constant:

More memory → more context for the current action → better decisions

More memory → eats into the LLM's context window → cost + limits

Memory is not free – it consumes the finite context window. So we manage it.

There are two types: **short-term** and **long-term**.

Short-term memory is limited to a single conversation thread. It is just a **list of previous chat messages**.

...

User → Agent: "Hi, I am Ankit."

User → Agent: "What is my name?"

Agent → User: "Your name is Ankit."

← only possible because msg 1 is still in the list

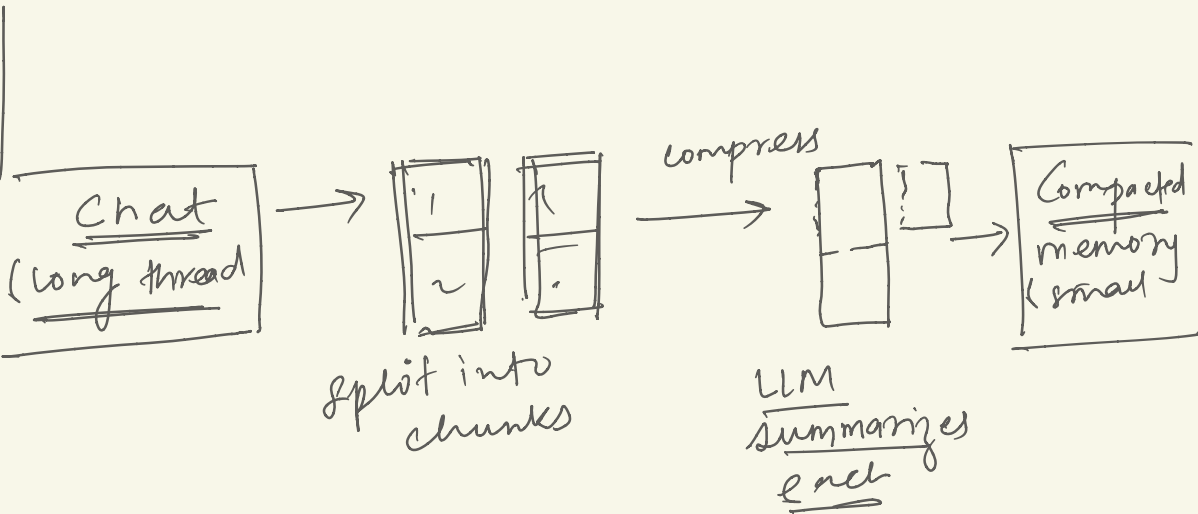
The LLM itself is stateless; the **list** is the memory.

→ Re-sending it each turn is what makes the conversation feel continuous.

The problem: the list grows forever

A long conversation makes the message list huge blowing past the context window (and the budget).

The fix is **compaction** – periodically summarize old chunks of the conversation using an LLM:



Trade off → some finer details lost
for staying within context
window.

56

Long-term memory – across conversations

> **Long-term memory is available across *multiple* conversation threads.** It stores **factual information** about a user – name, age, preferences – that should outlive any single chat.

Scene 1 – a visa form, in thread #1:

...

User: "Please fill this form for your visa application." 

Agent: "Sure. Please share the details needed. <form...>"

User: "Ankit Khowal, +91-..." ← user provides name + contact

Agent: "Form filled. Download the PDF here." 

...

Scene 2 – a brand-NEW thread, no long-term memory:

...

_____ NEW CONVERSATION THREAD _____

User: "I am hungry."

Agent: "I see – would you like me to suggest restaurants nearby?"

User: "Yes please."

Agent: "Restaurant #123, ..."

User: "Book #123 for 8 PM."

Agent: "Please share your name & contact number."

User: "I told you this earlier!" ← 😞 the agent forgot across threads

...

The agent ***did*** learn the name in Scene 1 – but ~~short-term~~ memory is ***per-thread***, so a new conversation starts blank. That's the gap long-term memory fills.

****Scene 2, replayed *with* long-term memory:****

...

User: "Book #123 for 8 PM."

Agent: "Booked under the name Ankit." ← name pulled from long-term memory

User: "Awesome."

...

WHEN is memory written?

Pattern A — End of session (batch / background)

Conversation ends



[Background "memory extraction" LLM call]



"User's name is Ankit. Prefers 8 PM bookings. Vegetarian."

Save to long-term store (DB / vector DB)

Pattern B — During the conversation (inline / hot-path)

After each turn (or every few turns), the system decides "is there anything worth saving?" and writes immediately.

Pattern C — Explicit / tool-triggered

The agent has a `save_memory ( fact )` tool and the LLM *decides* when to call it, or the user explicitly says "remember that I'm allergic to nuts."

WHEN a conversation starts, do we pull everything?

Strategy 1 — Load all

If a user has ~10–20 short facts, you *can* just inject all of them into the system prompt.

SSySSysaboutnameHyderbadcity=Hyderbadity=Hyderbadity=Hyderbadbadity=Hyderbadg=true, ..."

Strategy 2 — Retrieval (RAG) — the real answer for scale

When memory grows large (hundreds/thousands of facts, plus episodic history), you do **semantic retrieval**: embed the memories as vectors, and at the start of each turn, fetch only the top-K relevant ones for the *current* query.

Strategy 3 — Tiered memory (the "OS analogy")

MemGPT popularized treating memory like a computer's RAM vs. disk:

- **Core/working memory** → always-in-context tiny profile (loaded every time).
- **Archival memory** → huge store on "disk," pulled in via retrieval tool calls only when needed.

The agent itself decides "I need to look something up" and pages it in, like a CPU fetching from disk.

Putting it together — the typical production lifecycle

During chat
every turn:

1. embed user message
2. retrieve top-K relevant long-term memories (READ)
3. + always-loaded small user profile
4. + short-term memory (this thread's messages)
5. → send to LLM
6. (optionally) extract & save new facts (WRITE)

session ends

Background job

summarize transcript → extract durable facts
→ dedupe against existing → upsert to store (WRITE)